1. AI 配置页：读取本地配置并自动回填

代码位置: frontend/src/app/app/dashboard/ai/page.tsx:95-150

- 该段负责从 /api/local-config 读取当前服务商配置，并将 API Key、模型、Endpoint 回填到界面。
- 如果本地没有可用 Key，会显示明确错误提示，避免误用空配置发起解析。

```
 95:   const loadLocalProviderConfig = useCallback(async () => {
 96:     setLocalConfigLoading(true);
 97:     setLocalConfigError("");
 98:
 99:     try {
100:       const response = await fetch("/api/local-config", {
101:         method: "POST",
102:         headers: { "Content-Type": "application/json" },
103:         body: JSON.stringify({
104:           provider: currentModel,
105:           defaultEndpoint: meta.defaultEndpoint,
106:           defaultModel: meta.defaultModel || "",
107:         }),
108:       });
109:
110:       const json = (await response.json()) as {
111:         status?: string;
112:         message?: string;
113:         data?: LocalConfigPayload;
114:       };
115:
116:       if (!response.ok || json.status !== "ok" || !json.data) {
117:         throw new Error(
118:           json.message || t("dashboard.settings.ai.localConfigReadFailed")
119:         );
120:       }
121:
122:       const data = json.data;
123:       const localApiKey = data.apiKey?.trim() || "";
124:       if (localApiKey) {
125:         setProviderField(currentModel, "apiKey", localApiKey);
126:       }
127:       setProviderField(
128:         currentModel,
129:         "modelId",
130:         data.modelId || meta.defaultModel || ""
131:       );
132:       setProviderField(currentModel, "apiEndpoint", data.apiEndpoint || "");
133:       setProviderField(currentModel, "endpointMode", data.endpointMode || "official");
134:       setLocalConfigSources(data.source || []);
135:       setLocalConfigHasApiKey(Boolean(localApiKey));
136:
137:       if (!localApiKey) {
138:         setLocalConfigError(t("dashboard.settings.ai.localConfigKeyMissing"));
139:       }
140:     } catch (error) {
141:       setLocalConfigHasApiKey(false);
142:       setLocalConfigError(
143:         error instanceof Error
144:           ? error.message
145:           : t("dashboard.settings.ai.localConfigReadFailed")
146:       );
147:     } finally {
148:       setLocalConfigLoaded(true);
149:       setLocalConfigLoading(false);
150:     }
```

2. 导入面板：解析重试与 401 立刻停止

代码位置: frontend/src/components/agent/ResumeImportPanel.tsx:173-223

- 该段最多重试 5 次，选择置信度最高的解析结果。
- 一旦识别到 invalid api key / unauthorized，会立即停止重试并提示用户更新对应服务商 API Key。

```
173:       const maxAttempts = 5;
174:       let bestParseJson: any = null;
175:       let bestConfidence = -1;
176:       let lastErrorMessage = "";
177:       let shouldStopRetry = false;
178:
179:       for (let attempt = 1; attempt <= maxAttempts && !shouldStopRetry; attempt += 1) {
180:         const parseRes = await fetch("/api/agent/parse", {
181:           method: "POST",
182:           headers: { "Content-Type": "application/json" },
183:           body: JSON.stringify({
184:             session_id: uploadJson.session_id,
185:             ...parseCreds,
186:           }),
187:         });
188:         const parseJson = await parseRes.json();
189:
190:         if (parseJson.profile) {
191:           const confidence = Number(parseJson.confidence) || 0;
192:           if (confidence > bestConfidence) {
193:             bestConfidence = confidence;
194:             bestParseJson = parseJson;
195:           }
196:           if (confidence >= 1) {
197:             break;
198:           }
199:           continue;
200:         }
201:
202:         lastErrorMessage = parseJson.message || parseJson.detail || "Parse failed";
203:         const authLikeError =
204:           /invalid api key|unauthorized|authenticationerror|status_code=401/i.test(
205:             lastErrorMessage
206:           );
207:         if (authLikeError) {
208:           shouldStopRetry = true;
209:           const providerName = selectedModel.toUpperCase();
210:           lastErrorMessage = `□□ ${providerName} □ API Key □□□□□□□□□□“AI □□□ -> ${providerName}”□□□□□□ Key □□□□`;
211:         }
212:       }
213:
214:       if (bestParseJson?.profile) {
215:         setParseResult({
216:           profile: bestParseJson.profile,
217:           confidence: bestParseJson.confidence || 0,
218:           missing_fields: bestParseJson.missing_fields || [],
219:           open_questions: bestParseJson.open_questions || [],
220:         });
221:       } else {
222:         throw new Error(lastErrorMessage || "Parse failed");
223:       }
```

3. 本地配置路由：不同服务商的环境变量别名

代码位置: frontend/src/routes/api/local-config.ts:45-75

- 该段定义各 Provider 的 API Key / Base URL / Model 别名映射。
- 仅 OpenAI/Codex 允许泛化别名，降低错误读取其它服务商配置的风险。

```
45: const GENERIC_API_KEY_ALIASES = ["OPENAI_API_KEY", "API_KEY"];
46: const GENERIC_BASE_URL_ALIASES = ["OPENAI_BASE_URL", "BASE_URL", "API_BASE_URL"];
47: const GENERIC_MODEL_ALIASES = ["OPENAI_MODEL", "MODEL"];
48:
49: function allowGenericAliases(provider: AIModelType): boolean {
50:   return provider === "openai" || provider === "codex";
51: }
52:
53: const PROVIDER_ALIAS_MAP: Record<AIModelType, ProviderAliasConfig> = {
54:   deepseek: {
55:     apiKeys: ["DEEPSEEK_API_KEY"],
56:     baseUrls: ["DEEPSEEK_BASE_URL"],
57:     models: ["DEEPSEEK_MODEL"],
58:   },
59:   doubao: {
60:     apiKeys: ["DOUBAO_API_KEY", "ARK_API_KEY", "VOLCENGINE_API_KEY"],
61:     baseUrls: ["DOUBAO_BASE_URL", "ARK_BASE_URL", "VOLCENGINE_BASE_URL"],
62:     models: ["DOUBAO_MODEL", "ARK_MODEL"],
63:   },
64:   openai: {
65:     apiKeys: ["OPENAI_API_KEY", "OPENAI_AUTH_TOKEN"],
66:     baseUrls: ["OPENAI_BASE_URL"],
67:     models: ["OPENAI_MODEL", "GPT_MODEL"],
68:     tomlProviders: ["openai"],
69:   },
70:   codex: {
71:     apiKeys: ["OPENAI_API_KEY", "CODEX_API_KEY"],
72:     baseUrls: ["OPENAI_BASE_URL", "CODEX_BASE_URL"],
73:     models: ["CODEX_MODEL", "OPENAI_MODEL"],
74:     tomlProviders: ["codex", "openai"],
75:   },
```

4. 本地配置路由：防止跨服务商凭证泄漏

代码位置: frontend/src/routes/api/local-config.ts:296-356

- 该段按 provider 候选读取 TOML 配置；读取 auth.json 时仅在 provider=codex 才使用。
- 目的是避免把 codex 认证信息错误注入 OpenAI 等其它服务商导致鉴权失败。

```
296:     const providerCandidates = Array.from(
297:       new Set([provider, ...(providerAliases.tomlProviders || [])])
298:     );
299:
300:     for (const tomlProvider of providerCandidates) {
301:       const sectionEndpoint = findTomlStringInSection(
302:         codexTomlRaw,
303:         `model_providers.${tomlProvider}`,
304:         "base_url"
305:       );
306:       if (sectionEndpoint.trim()) {
307:         apiEndpoint = sectionEndpoint.trim();
308:         break;
309:       }
310:     }
311:
312:     const configuredProvider = findTomlString(codexTomlRaw, "model_provider")
313:       .trim()
314:       .toLowerCase();
315:     const configuredModel = findTomlString(codexTomlRaw, "model").trim();
316:     if (
317:       configuredModel &&
318:       configuredProvider &&
319:       providerCandidates.includes(configuredProvider)
320:     ) {
321:       modelId = configuredModel;
322:     }
323:   }
324:
325:   if (codexAuthRaw) {
326:     // Avoid leaking codex auth into other providers (e.g. OpenAI),
327:     // because auth.json may not contain a valid API key for them.
328:     if (provider === "codex") {
329:       sources.push(codexAuthPath);
330:       const authRecord = toEnvRecord(parseJsonRecord(codexAuthRaw));
331:
332:       if (!apiKey) {
333:         apiKey = pickFirstValue(authRecord, keyAliases);
334:       }
335:
336:       if (!modelId) {
337:         modelId = pickFirstValue(authRecord, modelAliases) || modelId;
338:       }
339:     }
340:   }
341:
342:   if (claudeSettingsRaw) {
343:     sources.push(claudeSettingsPath);
344:     const settingsJson = parseJsonRecord(claudeSettingsRaw);
345:     const settingsEnv = toEnvRecord(settingsJson.env);
346:     const settingsTopLevel = toEnvRecord(settingsJson);
347:     const mergedEnv = { ...settingsTopLevel, ...settingsEnv };
348:
349:     if (!apiKey) {
350:       apiKey = pickFirstValue(mergedEnv, keyAliases);
351:     }
352:
353:     const endpointFromEnv = pickFirstValue(mergedEnv, endpointAliases);
354:     const useEnvEndpoint =
355:       normalizeEndpoint(apiEndpoint) === normalizeEndpoint(defaultEndpoint);
356:     if (endpointFromEnv && useEnvEndpoint) {
```

5. 后端解析：置信度重计算规则

代码位置: backend/services/parser_service.py:1086-1133

- 该段按姓名/联系方式/教育/工作/项目/技能完整度计算综合分数。
- 当结构完整且非 fallback 解析时，直接返回 100%（1.0）。

```
1086:     def _recalculate_confidence(
1087:         self,
1088:         parsed: dict,
1089:         raw_text: str,
1090:         missing_fields: list[str],
1091:         unmapped_lines: list[str],
1092:     ) -> float:
1093:         metadata = parsed.get("metadata")
1094:         if not isinstance(metadata, dict):
1095:             metadata = {}
1096:
1097:         base_confidence = metadata.get("extraction_confidence")
1098:         try:
1099:             base_value = float(base_confidence)
1100:         except (TypeError, ValueError):
1101:             base_value = 0.82
1102:
1103:         basics = parsed.get("basics", {}) if isinstance(parsed.get("basics"), dict) else {}
1104:         has_name = bool(self._safe_text(basics.get("name")))
1105:         has_contact = bool(
1106:             self._safe_text(basics.get("phone")) or self._safe_text(basics.get("email"))
1107:         )
1108:         has_education = bool(parsed.get("education"))
1109:         has_work = bool(parsed.get("work_experience"))
1110:         has_project = bool(parsed.get("research"))
1111:         has_skill = self._has_any_skill(parsed)
1112:
1113:         checks = [has_name, has_contact, has_education, has_work, has_project, has_skill]
1114:         completeness = sum(1 for item in checks if item) / len(checks)
1115:
1116:         # Full structured hit should be shown as 100%.
1117:         if completeness >= 1.0 and not self._is_fallback_parse(metadata):
1118:             return 1.0
1119:
1120:         raw_line_count = len(
1121:             [line for line in raw_text.splitlines() if self._safe_text(line)]
1122:         )
1123:         unmapped_ratio = len(unmapped_lines) / max(raw_line_count, 1)
1124:         unmapped_penalty = min(0.05, unmapped_ratio * 0.05)
1125:         missing_penalty = min(0.12, len(missing_fields) * 0.02)
1126:
1127:         score = max(base_value, completeness)
1128:         score = score - unmapped_penalty - missing_penalty
1129:
1130:         if self._is_fallback_parse(metadata):
1131:             score = min(score, 0.35)
1132:
1133:         return round(max(0.0, min(score, 1.0)), 3)
```

6. 后端解析：输出最终 confidence/missing_fields

代码位置: backend/services/parser_service.py:1413-1433

- 该段把重算后的 confidence 写回 metadata，并在 API 返回体中输出 profile/confidence/missing_fields/open_questions。
- 前端导入页据此展示置信度并决定是否继续人工补充。

```
1413:         unmapped_lines = self._find_unmapped_lines(raw_text, parsed)
1414:         if unmapped_lines:
1415:             metadata["unmapped_lines"] = unmapped_lines
1416:
1417:         metadata["open_questions"] = self._unique_str_list(open_questions)
1418:
1419:         confidence_value = self._recalculate_confidence(
1420:             parsed=parsed,
1421:             raw_text=raw_text,
1422:             missing_fields=metadata.get("missing_fields", []),
1423:             unmapped_lines=unmapped_lines,
1424:         )
1425:         metadata["extraction_confidence"] = confidence_value
1426:
1427:         return {
1428:             "profile": parsed,
1429:             "confidence": metadata.get("extraction_confidence", 0.8),
1430:             "raw_text": raw_text,
1431:             "missing_fields": metadata.get("missing_fields", []),
1432:             "open_questions": metadata.get("open_questions", []),
1433:         }
```

7. 前端映射：先清理 AI 临时分组并重建菜单

代码位置: frontend/src/lib/profileConverter.ts:1695-1771

- 该段先删除历史 AI 自动分组，避免重复模块残留。
- 只按模板标准模块（basic/skills/experience/projects/education）恢复菜单并排序。

```
1695: function removeAiSections(
1696:   existing?: Partial<ResumeData>
1697: ): {
1698:   menuSections: ResumeData["menuSections"];
1699:   customData: ResumeData["customData"];
1700: } {
1701:   const menuSections = (existing?.menuSections || []).filter(
1702:     (section) =>
1703:       section.id !== IMPORT_EXTRA_SECTION_ID &&
1704:       !section.id.startsWith(AI_SECTION_PREFIX)
1705:   );
1706:
1707:   const customData = Object.fromEntries(
1708:     Object.entries(existing?.customData || {}).filter(
1709:       ([sectionId]) =>
1710:         sectionId !== IMPORT_EXTRA_SECTION_ID &&
1711:         !sectionId.startsWith(AI_SECTION_PREFIX)
1712:     )
1713:   ) as ResumeData["customData"];
1714:
1715:   return { menuSections, customData };
1716: }
1717:
1718: function getSectionSortRank(sectionId: string): number {
1719:   switch (sectionId) {
1720:     case "basic":
1721:       return 0;
1722:     case "skills":
1723:       return 1;
1724:     case "experience":
1725:       return 2;
1726:     case "projects":
1727:       return 3;
1728:     case "education":
1729:       return 4;
1730:     default:
1731:       if (sectionId.startsWith(AI_SECTION_PREFIX)) return 5;
1732:       if (sectionId === IMPORT_EXTRA_SECTION_ID) return 6;
1733:       return 999;
1734:   }
1735: }
1736:
1737: function ensureMenuSections(
1738:   existing: Partial<ResumeData> | undefined,
1739:   hasSectionData: Record<string, boolean>
1740: ): ResumeData["menuSections"] {
1741:   const isEnglish = inferIsEnglish(existing);
1742:   const current = [...(existing?.menuSections || [])];
1743:   const byId = new Map(current.map((section) => [section.id, { ...section }]));
1744:
1745:   for (const [id, hasData] of Object.entries(hasSectionData)) {
1746:     if (!hasData) continue;
1747:     const existingSection = byId.get(id);
1748:     if (existingSection) {
1749:       existingSection.enabled = true;
1750:       continue;
1751:     }
1752:     byId.set(id, {
1753:       id,
1754:       title: getSectionTitle(id, isEnglish),
1755:       icon: "*",
1756:       enabled: true,
1757:       order: 999,
1758:     });
1759:   }
1760:
1761:   const sorted = Array.from(byId.values()).sort((a, b) => {
1762:     const ar = getSectionSortRank(a.id);
1763:     const br = getSectionSortRank(b.id);
1764:     if (ar !== br) return ar - br;
1765:     return (a.order ?? 999) - (b.order ?? 999);
1766:   });
1767:
1768:   return sorted.map((section, idx) => ({
1769:     ...section,
1770:     order: idx,
1771:   }));
```

8. 前端映射：工作经历字段精确落位

代码位置: frontend/src/lib/profileConverter.ts:2055-2130

- 该段把 work_experience 映射到 company/position/date/details，确保时间与职责进入对应输入框。
- 职责统一转成 HTML 列表，避免所有文本堆到单个字段。

```
2055: export function fromResumeProfile(
2056:   profile: ResumeProfile,
2057:   existing?: Partial<ResumeData>
2058: ): Partial<ResumeData> {
2059:   const isEnglish = inferIsEnglish(existing);
2060:   const { menuSections: cleanedMenuSections, customData: cleanedCustomData } =
2061:     removeAiSections(existing);
2062:
2063:   const basic = {
2064:     name: profile.basics.name || "",
2065:     title: profile.basics.target_role || "",
2066:     email: profile.basics.email || "",
2067:     phone: profile.basics.phone || "",
2068:     location: "",
2069:     birthDate: "",
2070:     employementStatus: "",
2071:     photo: "",
2072:     customFields: [],
2073:     icons: {},
2074:     githubKey: "",
2075:     githubUseName: "",
2076:     githubContributionsVisible: false,
2077:     ...(existing?.basic || {}),
2078:     // □□□□□□
2079:     ...(profile.basics.name ? { name: profile.basics.name } : {}),
2080:     ...(profile.basics.email ? { email: profile.basics.email } : {}),
2081:     ...(profile.basics.phone ? { phone: profile.basics.phone } : {}),
2082:     ...(profile.basics.target_role ? { title: profile.basics.target_role } : {}),
2083:   };
2084:
2085:   const education = (profile.education || []).map((rawEdu, i) => {
2086:     const edu = asObject(rawEdu);
2087:     return {
2088:       id: `edu_${i}`,
2089:       school: pickFirstText(edu.school, edu.university, edu.college),
2090:       major: pickFirstText(edu.major, edu.field, edu.specialization),
2091:       degree: pickFirstText(edu.degree_type, edu.degree),
2092:       startDate: formatDateForSingleField(
2093:         parseDateToken(pickFirstText(edu.start_date, edu.start, edu.from))
2094:       ),
2095:       endDate: formatDateForSingleField(
2096:         parseDateToken(pickFirstText(edu.end_date, edu.end, edu.to))
2097:       ),
2098:       gpa: pickFirstText(edu.gpa),
2099:       description: "",
2100:       visible: true,
2101:     };
2102:   });
2103:
2104:   let experience = (profile.work_experience || []).map((rawExp, i) => {
2105:     const exp = asObject(rawExp);
2106:     const startDate = pickFirstText(
2107:       exp.start_date,
2108:       exp.start,
2109:       exp.from,
2110:       pickPeriodDate(exp.periods, "start")
2111:     );
2112:     const endDate = pickFirstText(
2113:       exp.end_date,
2114:       exp.end,
2115:       exp.to,
2116:       pickPeriodDate(exp.periods, "end")
2117:     );
2118:     const responsibilities = normalizeResponsibilitiesFromAny(exp);
2119:
2120:     return {
2121:       id: `exp_${i}`,
2122:       company: pickFirstText(exp.company, exp.organization, exp.employer),
2123:       position: pickFirstText(exp.role, exp.title, exp.position, exp.job_title),
2124:       date: normalizeDateRangeFromFields(startDate, endDate, isEnglish),
2125:       details: responsibilitiesToHtml(responsibilities),
2126:       visible: true,
2127:     };
2128:   });
2129:   experience = mergeDuplicateExperienceItems(experience);
2130:
```

9. 前端映射：项目与经历纠偏、去除无效补充项

代码位置: frontend/src/lib/profileConverter.ts:2131-2189

- 该段将误判为项目的经历回迁并去重，删除空的补充项。
- 最终只保留真实项目数据，避免“多出不该有模块”或内容堆叠。

```
2131:   let projects = (profile.research || []).map((rawRes, i) => {
2132:     const res = asObject(rawRes);
2133:     const description = pickFirstText(res.description, res.summary);
2134:     return {
2135:       id: `proj_${i}`,
2136:       name: pickFirstText(res.title, res.name),
2137:       role: pickFirstText(res.contributions, res.role),
2138:       date: normalizeDateRangeFromFields(
2139:         pickFirstText(res.start_date, res.start, res.from),
2140:         pickFirstText(res.end_date, res.end, res.to),
2141:         isEnglish
2142:       ),
2143:       description: description ? `<p>${description}</p>` : "",
2144:       visible: true,
2145:     };
2146:   });
2147:
2148:   ({ experience, projects } = migrateProjectLikeExperienceItems(
2149:     experience,
2150:     projects,
2151:     isEnglish
2152:   ));
2153:
2154:   experience = pruneEmptySupplementExperiences(
2155:     mergeDuplicateExperienceItems(experience)
2156:   );
2157:   projects = absorbSupplementProjectDetails(
2158:     mergeDuplicateProjects(pruneEmptySupplementProjects(projects))
2159:   );
2160:   const skillContent = skillsCategoriesToHtml(profile.skills || {});
2161:   const customData = {
2162:     ...cleanedCustomData,
2163:   };
2164:
2165:   const existingForMenu: Partial<ResumeData> = {
2166:     ...(existing || {}),
2167:     menuSections: [...cleanedMenuSections],
2168:   };
2169:
2170:   const hasSectionData: Record<string, boolean> = {
2171:     basic: true,
2172:     skills: !!skillContent.trim(),
2173:     experience: experience.length > 0,
2174:     projects: projects.length > 0,
2175:     education: education.length > 0,
2176:   }
2177:
2178:   const menuSections = ensureMenuSections(existingForMenu, hasSectionData);
2179:
2180:   return {
2181:     ...(existing || {}),
2182:     basic: basic as ResumeData["basic"],
2183:     education,
2184:     experience,
2185:     projects,
2186:     skillContent,
2187:     customData,
2188:     menuSections,
2189:   };
```